

Listen in Python – oder: Wie man seine Daten hübsch verpackt

Einleitung

Listen gehören zu den grundlegenden Datenstrukturen in Python. Sie erlauben es, mehrere Werte in geordneter Weise zu speichern, auf sie zuzugreifen, sie zu verändern und systematisch zu verarbeiten. Durch ihre Flexibilität und einfache Handhabung sind Listen für viele algorithmische Aufgaben ein geeignetes Werkzeug. In diesem Kapitel betrachten wir die wesentlichen Aspekte von Listen, darunter ihre Erzeugung, den Zugriff auf Elemente, typische Bearbeitungsmethoden und den Einsatz in Schleifen und sogenannten *List Comprehensions*.

1. Erzeugung von Listen

Eine Liste in Python wird durch eckige Klammern notiert. Die enthaltenen Elemente werden durch Kommata getrennt:

```
zahlen = [1, 2, 3, 4]
```

Listen können unterschiedliche Datentypen enthalten, etwa ganze Zahlen, Zeichenketten oder Wahrheitswerte:

```
gemischt = [42, "Hallo", True]
```

Auch leere Listen sind möglich:

```
leer = []
```

Alternativ lassen sich Listen über Konstruktorfunktionen wie `list()` erstellen:

```
aus_string = list("abc")    # ['a', 'b', 'c']
aus_range = list(range(5))  # [0, 1, 2, 3, 4]
```

2. Zugriff auf Elemente

Listen sind geordnet, das heißt, ihre Elemente haben eine feste Reihenfolge. Der Zugriff erfolgt über sogenannte *Indizes*, beginnend bei 0:

```
zahlen[0]    # ergibt 1
zahlen[-1]   # ergibt 4 (letztes Element)
```

Mit *Slicing* kann man Teilstücke einer Liste extrahieren:

```
zahlen[1:3]  # ergibt [2, 3]
zahlen[:2]   # ergibt [1, 2]
zahlen[2:]   # ergibt [3, 4]
```

Dabei gilt stets: der Startindex ist inklusive, der Endindex exklusiv.

3. Veränderung von Listen

Listen sind *veränderbare* (mutable) Objekte. Das bedeutet, dass ihre Inhalte nachträglich geändert werden können:

```
zahlen[2] = 99          # ersetzt das dritte Element durch 99
zahlen.append(5)         # fügt 5 am Ende an
zahlen.insert(1, 42)     # fügt 42 an Position 1 ein
zahlen.remove(99)        # entfernt erstes Vorkommen von 99
letztes = zahlen.pop()   # entfernt und gibt das letzte Element zurück
```

`append` ist die einfachste Methode, um Listen zu erweitern. Mit `insert` kann man gezielt einfügen, während `remove` und `pop` der Entfernung von Elementen dienen.

4. Iteration über Listen

Ein zentrales Anwendungsfeld von Listen ist die systematische Verarbeitung ihrer Elemente. Dies geschieht meist mit einer `for`-Schleife:

```
for zahl in zahlen:
    print(zahl)
```

Wenn zusätzlich die Position des Elements benötigt wird, kann `enumerate` verwendet werden:

```
for i, zahl in enumerate(zahlen):  
    print(i, zahl)
```

5. List Comprehensions

Python erlaubt eine besonders kompakte Notation zur Erstellung neuer Listen auf Basis vorhandener Daten: *List Comprehensions*.

```
quadrade = [x**2 for x in range(5)]
```

Diese Zeile erzeugt eine Liste der Quadratzahlen von 0 bis 4. Bedingungen lassen sich integrieren:

```
gerade_quadrade = [x**2 for x in range(10) if x % 2 == 0]
```

So entstehen neue Listen durch Kombination von Iteration, Selektion und Transformation – in einer Zeile.

6. Nützliche Funktionen

Für viele Aufgaben bietet Python eingebaute Funktionen zur Verarbeitung von Listen:

```
len(zahlen)      # Länge der Liste  
sum(zahlen)      # Summe aller Elemente  
min(zahlen)      # kleinstes Element  
max(zahlen)      # größtes Element  
zahlen.count(4)  # Anzahl der Vorkommen von 4  
zahlen.index(3)  # Position des ersten Vorkommens von 3
```

`len` und `sum` sind hilfreich bei numerischen Listen, `count` und `index` bei der Analyse von Inhalten.

7. Kopieren und Kombinieren

Listen lassen sich mit dem Pluszeichen verbinden:

```
a = [1, 2]  
b = [3, 4]  
c = a + b      # [1, 2, 3, 4]
```

Achtung: Eine Zuweisung wie `b = a` erzeugt keine Kopie, sondern eine zweite Referenz auf dieselbe Liste. Um eine Kopie zu erstellen, kann man `copy()` oder ein Slice verwenden:

```
b = a.copy()
# oder
b = a[:]
```

8. Beispielaufgabe

Gegeben ist eine Liste von Zahlen. Gesucht ist eine Liste, die die Quadrate aller geraden Zahlen enthält.

```
zahlen = [1, 2, 3, 4, 5, 6]
ergebnis = [x**2 for x in zahlen if x % 2 == 0]
print(ergebnis) # Ausgabe: [4, 16, 36]
```

Diese Aufgabe illustriert die kombinierte Verwendung von Iteration, Bedingung und Transformation in einer List Comprehension.